Low Level Design (LLD)

Regional Sales Data Analysis

Submitted by	Hari A Menon Harisreeram Rajesh Liyan Sebastian Lidhun Krishna M Anupam Sawanth S
Document Version	1
Last Revised Date	21 - June-2026

Contents

1. Introduction

1.1. What is a Low-Level design document?

The goal of the Low-Level Design (LLD) document is to provide the internal logical design, data modeling mechanics, and architectural rules for the Sales Performance & Regional Profitability Analytics System. The LLD describes the detailed layout of the system, including table relationships, specific data cleaning functions, calculation metrics, and structural interface modules. It explains how the automated ingestion engine, transformation layers, and reporting components interact so that developers can directly deploy or modify the production framework.

1.2. Scope

Low-Level Design (LLD) is a component-level design process used to define the software architecture, relational data structures, and engineering workflow of the business intelligence platform. The system processes multi-channel US transactional records to map performance trends, flag business risks like client concentration, optimize inventory plans by evaluating seasonal fluctuations, and deliver interactive, no-code visual discovery assets directly to corporate executive stakeholders.

2.Process Flow

3.Architecture Description

3.1 Data Description

The system utilizes a multi-entity transactional structure sourcing historical retail operations, tracking a massive central log of 64,104 core sales order entries. The framework blends this transactional layer against supporting domain dimensions including customer demographic indexes, localized territory population breakdowns, regional geographic positioning fields, and active target budget parameters.

3.2 Data Ingestion

Data ingestion establishes the foundation of the pipeline, pulling raw transactional components from local directories or structured file layers into the operational Power Query workspace engine. Entities are mapped explicitly as tabular staging tables to preserve initial records prior to parsing or normalization modifications

3.3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis evaluates categorical feature counts, identifies missing fields, and maps spatial patterns to discover market dynamics. Both univariate metrics and multivariate trend charts are engineered to uncover relationships, such as how sales channel distributions directly skew final net profit margins across states.

3.4. Data Validation

Data validation ensures structural integrity prior to analytical compilation. Integrity checks confirm that vital unique identifier vectors (Order Number, Customer Index, Product Index) map properly, verifying that data types align perfectly with predefined analytical models to stop corrupted or mismatched entries from breaking down calculation paths.

3.5. Data Cleaning and Validation

Data cleaning routines actively remove risk vectors from ingested tables. Duplicated actions are identified and eliminated, data fields are checked to resolve missing parameters, and disparate data metrics are scrubbed to conform to explicit structured formats across all source layers.

3.6. Data Preprocessing

Data preprocessing handles type assignments, structural formatting, and data cleansing operations. Text entries are normalized for layout symmetry, and numbers are set to clean currencies or integers, ensuring that the processed records maintain total functional readiness for high-speed computation modules.

3.7. Feature Engineering

Feature engineering designs operational business columns by evaluating transaction vectors. Key calculations are computed directly across row records using the following logic equations:

Revenue:

Revenue = Order Quantity X Unit Price

Total Cost:

Total Cost = Order Quantity X Total Unit Cost

Profit:

Profit = Revenue - Total Cost

Profit Margin (%):

Profit Margin = (Profit/Revenue) X100

3.8. Data Transformation

Data transformation merges structural datasets into an optimized analytics architecture. Row structures are explicitly shaped, numeric columns are scaled into clean granular levels, and lookup vectors are constructed to link separate regional datasets together efficiently

3.9. Relational Data Modelling

The system organizes isolated source datasets into a high-performance, single-point **Star Schema** modeling configuration. The Sales Orders log functions as the central Fact table, interconnected via precise **One-to-Many (1….*) relationships** out to peripheral dimension nodes (Customers, Products, Regions), minimizing database indexing loops during active dashboard cross-filtering.

3.10. Business Logic Layer & DAX Metrics

Complex business insights are handled using Data Analysis Expressions (DAX) to build dynamic metrics. These measures handle fast, user-driven aggregations, compute running totals over time, and evaluate variations between actual sales figures and preset annual budgets.

3.11. UI Dashboard Components (Executive Overview)

The front-end user portal builds explicit interactive summaries across specific dedicated views. The **Executive Overview & Trends** view provides immediate high-level clarity via primary KPI visualization tools and detailed interactive dashboards:

Key Financial Elements: Tracks $1.2bn Total Revenue, $461.8M Net Profit, and a 37.36% Average Profit Margin.

Visualizations: Tracks a monthly revenue chart highlighting seasonality peaks (showing clear spikes in May), paired with a multi-variable price scatter index map.

3.12. UI Dashboard Components (Product & Channel)

The **Product & Channel Performance** interface isolates inventory asset trends and route efficiency metrics:

Product Breakdown: Ranks top-grossing inventory items (led by Product 26 at $117M and Product 27 at $109M).

Channel Analytics: Tracks operational revenue splits using donut charts, highlighting that the Wholesale channel drives a dominant 54.06% ($668.2M) of gross inflows

4. Unit Test Cases

Test Case Description	Pre-Requisite	Expected Result
Verify total cost calculation logic.	Total cost column is calculate using the formula: **order quantity X unit cost**	The newly created total cost column must perfectly match order quantity multiplied by unit cost.
Verify profit calculation logic.	Profit column is calculated using the formula: **Revenue – Total cost**	The created profit value must exactly equal line total minus computed total cost.
Mathematical Feature Sync.	Aggregation calculations active across rows.	Computed features match the backend validation metrics perfectly
Cross-Filter Dynamic Update.	Interactive report filters active on dashboard page.	Selecting an individual sector (e.g., "Wholesale") instantly isolates and filters all related map and KPI elements.